Experimental Comparison of 2-bit Vector Quantizers

Diego Manuel Maldonado Nunez
Student ID: 202620726

July 2026

1 Introduction

Vector quantization is a compression method which can be used to compress real-value vectors into a smaller representation, by using what is called a codebook. Instead of storing every value of the original vector with full precision, a quantizer maps the vector to a very similar vector in the codebook, and gives an index. The dequantizer then takes this index, and retrieves the similar vector. This of course leads to a loss in accuracy. Commonly this is measured using what is called SQNR, with ther following formula:

$$\mathrm{SQNR}_{\mathrm{bits}}(x, \hat{x}) = \frac{1}{2} \log_2 \left(\frac{\frac{1}{d} \sum_{i=1}^{d} x_i^2}{\frac{1}{d} \sum_{i=1}^{d} (x_i - \hat{x}_i)^2} \right).$$

As I was assisting a research in my laboratory, I came across an algorithm that is called Leech Lattice Vector Quantization[4]. This algorithm uses the Leech Lattice, a lattice wich has exceptional symmetry, high minimum distance, and rich shell structure, which makes it well suited for generating spherical codes with fast encoding and decoding procedures.

Because there is no source code for this, I was tasked with reconstructing this algorithm, which after complete, needs to be benchmarked. It is also important to compare to other quantization techniques available in the literature.

In this experiment, we aim to determine which vector quantization technique is best for compressing the weights of a Large Language Model into a 2 bit representation. Because Large Language models often feature billions or trillions of parameters, or the "weights", the amount of memory used for them is often very big. Therefore, by using vector quantization techniques, it is possible to reduce the amount of memory being used when performing inference. Of course, because the weights are being compressed, there will be a reconstruction error and a decrease in quality. Also, using vector quantization increases the amount of operations to perform inference, as it is necessary to add additional steps to decompress the reconstruction from the codebook, which results in an increase of processing steps, and therefore the inference time is increased.

The four algorithms compared are:

- **Uniform scalar 2-bit**: as the name suggests, this reduces the original values to a scalar representation. It is the most naive version of quantization.
- **QuIP# E8P12**: an 2-bit vector quantizer based on the QuIP# E8 lattice codebook [2]. The E8P12 codebook quantizes 8-dimensional blocks with 16-bit indices, so each 24-dimensional

experimental block is represented by three E8P12 indices for an effective rate of 2 bits per dimension.

- **QTIP bitshift**: an additional quantization algorithm based on the bitshift codebook component from QTIP, which stands for Quantization with Trellises and Incoherence Processing [1]. QTIP uses trellis-coded quantization to support very high-dimensional quantization, and introduces a hardware-efficient bitshift trellis for fast decoding.
- **Leech Lattice Vector Quantization**: the previously mentioned technique.

The main question we want to answer is:

> Which 2-bit quantization algorithm gives the best trade-off between reconstruction quality, quantization time, and dequantization time?

The expected trade-off is that the Leech Lattice Vector Quantization may achieve better reconstruction quality, as mentioned in the paper. However, quantization and dequantization time are not reported, so it is to be seen how well the algorithms perform in these metrics.

2 Experiment Design

2.1 Variables

The independent variable is the quantization algorithm. It has four levels: Uniform scalar 2-bit, QuIP# E8P12, QTIP bitshift, and Leech Lattice Vector Quantization. The dependent variables are SQNR bits, quantize seconds, and dequantize seconds.

2.2 Metrics used in the experiment

2.2.1 SQNR bits

SQNR bits is the signal-to-quantization-noise ratio expressed in bits. Higher is better. This is the quality metric used for sample-size calculation, hypothesis testing, and interpretation. For an original vector and its dequantized reconstruction, SQNR bits measures the ratio between signal power and quantization-noise power, expressed in base-2 logarithmic units:

2.2.2 Quantize seconds

Quantize seconds is the wall-clock time needed to encode one vector. Lower is better. This measures the cost of producing the compressed representation, which matters when vectors must be encoded online or repeatedly during a larger pipeline. However, because this is only done once, it is of less significance for inference. It is still measured regardless, as quantizing would involve a lot of computation and should be measured.

2.2.3 Dequantize seconds

Dequantize seconds is the wall-clock time needed to decode one vector. Lower is better. This measures the cost of reconstructing vectors from their compressed representation. Because dequantization needs to be done when performing inference, it is of more importance than quantize seconds.

These three metrics were selected because they correspond to the practical trade-off in a quantization system: reconstruction quality, encoding cost, and decoding cost. Considering them together makes it possible to distinguish an algorithm that is accurate but expensive from one that is faster but less accurate.

2.3 Hypotheses

As we are comparing multiple quantization techniques, and measuring the difference in each metric, an ANOVA study is performed for each one:

- SQNR bits
- quantize time
- dequantize time

$$H_0 : \mu_1 = \mu_2 = \mu_3 = \mu_4$$
$$H_1 : \exists i, j \text{ such that } \mu_i \neq \mu_j$$

where μ_k is the mean of the results of performing quantization and dequantization for each metric. Since there are three, an ANOVA experiment is performed on each.

3 Data Collection Protocol

The experiment was implemented in `quantizer_sample_size_experiment.ipynb`. All input vectors were actual model-weight blocks from Qwen3-0.6B [3].

The Leech lattice is 24-dimensional, so each experimental block used 24 weights. Each block was formed by flattening a selected weight tensor and taking 24 consecutive weights starting at a sampled tensor-local block index. This was done randomly from each of the 28 layers of the model.

The experiment was conducted in two phases:

1. **Pilot experiment**: 30 paired blocks were collected. In each block, the same vector was quantized and dequantized by all four algorithms, creating the dependent variables.
2. **Final experiment**: the pilot experiment yielded the amount of samples to be used for the experiment. It consisted of 67 weight blocks from 24 distinct layers.

Each block consisted of one vector evaluated by every algorithm. The order of algorithms was randomized during timing to reduce ordering effects. Before timing, each algorithm was warmed up for 5 iterations. For GPU timing, CUDA synchronization was performed before and after each measured call, so asynchronous CUDA execution would not contaminate timing measurements.

Each CSV row records the phase, block id, sampled weight tensor, layer, tensor-local block index, method, SQNR bits, quantization time, dequantization time, the original vector, the dequantized vector, dimension, and seed. The final data used in this report is:

`results/quantizer_sample_size_experiment/seed0_llm_weight_blocks/final_experiment.csv`

4 Data Analysis Protocol

First, the pilot data was used to estimate the sample size for each dependent variable. A power calculation for a four-algorithm mean comparison was then performed. Two effect scenarios were evaluated:

- **Two levels symmetric**: two algorithms differ symmetrically around the grand mean.
- **One versus rest**: one algorithm differs from the other algorithms.

Following the ANOVA effects model, the group means can be written as

$$\mu_i = \mu + \tau_i, \qquad \sum_{i=1}^{4} \tau_i = 0,$$

where μ is the grand mean and τ_i is the effect of algorithm i. If δ is the minimum interesting difference, the two planned effect scenarios were:

$$\text{Two levels symmetric: } (\tau_1, \tau_2, \tau_3, \tau_4) = \left(-\frac{\delta}{2}, \frac{\delta}{2}, 0, 0\right)$$

$$\text{One versus rest: } (\tau_1, \tau_2, \tau_3, \tau_4) = \left(-\frac{3\delta}{4}, \frac{\delta}{4}, \frac{\delta}{4}, \frac{\delta}{4}\right)$$

For each metric and scenario, the minimum sample size per algorithm was calculated using the noncentral F distribution. The final experiment used the largest sample size from all of the runs. The target significance level was $\alpha = 0.05$ and the desired power was 0.80.

The minimum interesting differences selected were:

- SQNR bits: 0.10 bits.
- Quantize time: 0.1 seconds.
- Dequantize time: 0.01 seconds. Note how the dequantize time difference is smaller than quantize, as quantize is slower due to searching the vectors in the codebook.

4.1 Available Hardware and Software

The experiment was executed on the following machine:

- CPU: AMD Ryzen 9 5950X 16-Core Processor, 16 cores / 32 threads.
- RAM: 62 GiB system memory.
- GPU: NVIDIA GeForce RTX 3080, 10 GiB VRAM, compute capability 8.6.
- NVIDIA driver: 595.80.
- Operating system: Linux 7.0.13-200.fc44.x86_64.
- Python: 3.12.9.
- PyTorch: 2.12.1+cu130, CUDA 13.0 available.

- NumPy: 2.4.6, Pandas: 3.0.3, SciPy: 1.18.0.

5 Actual Experiment

5.1 Sample Size Calculation

Table 1 shows the pilot sample size calculation. The largest required sample size was 67 observations per algorithm, coming from the SQNR metric. For this reason, the final experiment used a sample size of 67.

Table 1: Pilot-based sample-size calculation.

Metric	Scenario	Pilot SD	Min. diff.	Required n
SQNR bits	Two levels symmetric	0.173003	0.10000	67
SQNR bits	One versus rest	0.173003	0.10000	45
Quantize seconds	Two levels symmetric	0.000253	0.10000	3
Quantize seconds	One versus rest	0.000253	0.10000	3
Dequantize seconds	Two levels symmetric	0.000054	0.01000	3
Dequantize seconds	One versus rest	0.000054	0.01000	3

5.2 Descriptive Results

The final experiment collected 67 paired Qwen3-0.6B weight blocks for each algorithm. These blocks were randomly sampled from 24 distinct MLP down-projection layers. Figure 1 shows the distributions of the main metrics.

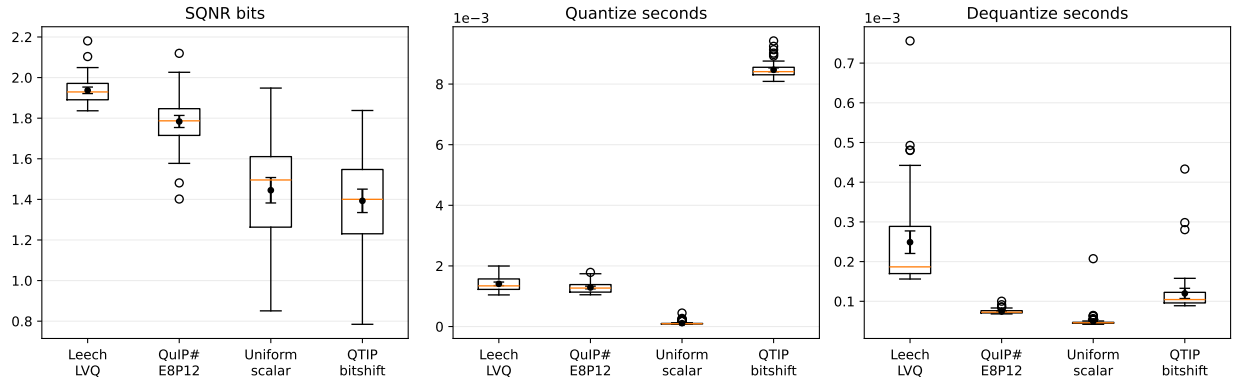

Figure 1: Final metric distributions for the four algorithms. Black points show the mean with 95% confidence intervals.

Table 2 shows the reconstruction quality. Leech Lattice Vector Quantization achieved the best reconstruction quality, with the highest SQNR bits, followed by QuIP# E8P12.

Table 2: Final reconstruction quality statistics.

Algorithm	SQNR mean	SQNR SD	SQNR 95% CI	Coef. variation
Leech Lattice Vector Quantization	1.937313	0.065726	[1.921281, 1.953345]	0.033927
QuIP# E8P12	1.783790	0.121918	[1.754052, 1.813528]	0.068348
Uniform scalar 2-bit	1.444994	0.256908	[1.382329, 1.507659]	0.177791
QTIP bitshift	1.392786	0.236477	[1.335105, 1.450468]	0.169787

Tables 3 and 4 summarize runtime. Uniform scalar quantization was the fastest method for both quantization and dequantization.

Table 3: Final quantize-time statistics.

Algorithm	Mean (s)	SD	95% CI
Uniform scalar 2-bit	0.000112	0.000059	[0.000098, 0.000127]
QuIP# E8P12	0.001289	0.000183	[0.001244, 0.001333]
Leech Lattice Vector Quantization	0.001409	0.000241	[0.001350, 0.001468]
QTIP bitshift	0.008469	0.000260	[0.008406, 0.008533]

Table 4: Final dequantize-time statistics.

Algorithm	Mean (s)	SD	95% CI
Uniform scalar 2-bit	0.000050	0.000020	[0.000045, 0.000054]
QuIP# E8P12	0.000074	0.000005	[0.000073, 0.000076]
QTIP bitshift	0.000120	0.000053	[0.000107, 0.000133]
Leech Lattice Vector Quantization	0.000249	0.000116	[0.000220, 0.000277]

From these results, we can already see that the Leech Lattice Vector Quantization technique yields a higher SQNR. In the case of runtime however, the uniform scalar, which is the simplest quantization technique, wins in quantizze and dequantize time.

5.3 Statistical Tests

The ANOVA tests rejected the null hypothesis for all three primary metrics. Table 5 shows strong evidence of algorithm differences.

Table 5: Global statistical tests on final data.

Metric	Test	Statistic	p-value
SQNR bits	ANOVA	164.90	1.37×10^{-53}
Quantize seconds	ANOVA	27469.27	4.34×10^{-259}
Dequantize seconds	ANOVA	124.96	1.91×10^{-45}

5.4 Normality Check

Table 6 shows the Shapiro-Wilk checks on ANOVA residuals. SQNR passed the normality check, but quantization-time and dequantization-time did not. Therefore, the ANOVA results for timing should be interpreted with caution.

Table 6: Shapiro-Wilk residual normality checks.

Metric	Test	p-value	Passes $\alpha = 0.05$
SQNR bits	Shapiro-Wilk residual normality	0.8389	Yes
Quantize seconds	Shapiro-Wilk residual normality	0.000107	No
Dequantize seconds	Shapiro-Wilk residual normality	6.66×10^{-16}	No

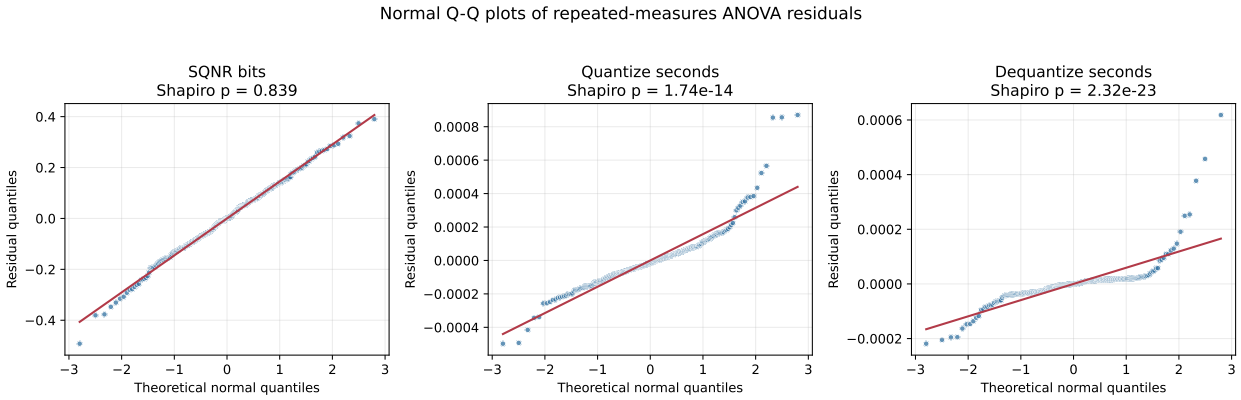

Figure 2: Normal Q-Q plots of ANOVA residuals for the three primary metrics.

In Figure 2, the SQNR aligns the reference line, which indicates normality for SQNR. However, the quantize and dequantize times dont align to the line, specially around the edges. This reinforces non normality for quantize and dequantize time.

5.5 Homoscedasticity Check

Figure 3 shows the ANOVA residuals against fitted values for each metric. This diagnostic checks whether the spread is similar across the value range.

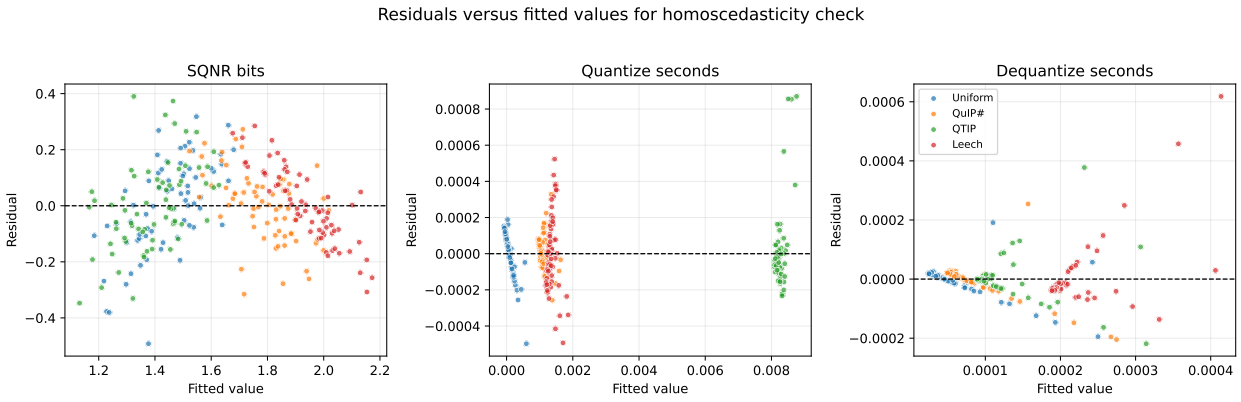

Figure 3: Residuals versus fitted values for checking homoscedasticity across the three primary metrics.

In Figure 3, SQNR has a larger absolute scale, but their spread is relatively similar across the value range. For the timing metrics, the values form distinct clusters by algorithm, and quantize time shows more unequal residual spread across algorithms. This means the equal-variance assumption is most questionable for quantize time, so the timing ANOVA p-values should be interpreted cautiously and supported by the non-parametric Friedman analysis.

5.6 Friedman Test

Because the timing residuals did not pass the normality check, a Friedman test was also performed as a non-parametric alternative to ANOVA.

Table 7: Friedman tests on data.

Metric	Test	Statistic	p-value
SQNR bits	Friedman test	148.88	4.59×10^{-32}
Quantize seconds	Friedman test	188.43	1.33×10^{-40}
Dequantize seconds	Friedman test	183.66	1.43×10^{-39}

The Friedman tests reject the null hypothesis for all three metrics. This results in a similar conclusion as ANOVA: the algorithms have significant differences in reconstruction quality, quantization time, and dequantization time.

5.7 Post-hoc Comparisons After Friedman Test

After the Friedman tests, pairwise Wilcoxon signed-rank tests were used as non-parametric post-hoc comparisons. Because there are three metrics and six pairwise algorithm comparisons per metric, the p-values were Bonferroni-corrected over 18 planned comparisons. The corrected significance threshold was:

$$\alpha_{\text{Bonferroni}} = \frac{\alpha}{m} = \frac{0.05}{18} = 0.002778,$$

where m is the number of planned comparisons.

Table 8: Bonferroni-corrected Wilcoxon signed-rank post-hoc comparisons.

Metric	Comparison	Wilcoxon statistic	Adjusted p-value	Better method
SQNR bits	Uniform vs QuIP#	49	1.77×10^{-10}	QuIP#
SQNR bits	Uniform vs QTIP	815	0.774	Uniform
SQNR bits	Uniform vs Leech	0	2.02×10^{-11}	Leech
SQNR bits	QuIP# vs QTIP	38	1.10×10^{-10}	QuIP#
SQNR bits	QuIP# vs Leech	142	8.51×10^{-9}	Leech
SQNR bits	QTIP vs Leech	0	2.02×10^{-11}	Leech
Quantize seconds	Uniform vs QuIP#	0	2.02×10^{-11}	Uniform
Quantize seconds	Uniform vs QTIP	0	2.02×10^{-11}	Uniform
Quantize seconds	Uniform vs Leech	0	2.02×10^{-11}	Uniform
Quantize seconds	QuIP# vs QTIP	0	2.02×10^{-11}	QuIP#
Quantize seconds	QuIP# vs Leech	180	3.76×10^{-8}	QuIP#
Quantize seconds	QTIP vs Leech	0	2.02×10^{-11}	Leech
Dequantize seconds	Uniform vs QuIP#	131	5.48×10^{-9}	Uniform
Dequantize seconds	Uniform vs QTIP	64	3.38×10^{-10}	Uniform
Dequantize seconds	Uniform vs Leech	1	2.11×10^{-11}	Uniform
Dequantize seconds	QuIP# vs QTIP	65	3.53×10^{-10}	QuIP#
Dequantize seconds	QuIP# vs Leech	59	2.73×10^{-10}	QuIP#
Dequantize seconds	QTIP vs Leech	170	2.56×10^{-8}	QTIP

The post-hoc comparisons showed the following ordering:

- **SQNR bits**: Leech Lattice Vector Quantization > QuIP# > Uniform > QTIP. Uniform had a higher mean than QTIP, but that pairwise difference was not significant after Bonferroni correction.
- **Quantize time**: Uniform < QuIP# < Leech Lattice Vector Quantization < QTIP.
- **Dequantize time**: Uniform < QuIP# < QTIP < Leech Lattice Vector Quantization.

6 Discussion

The results show that Leech Lattice Vector quantization is the best when it comes to reconstruction quality after quantization, however it falls short on time against other algorithms. This means that when selecting a 2-bit quantization technique, it is important to evaluate whether it is better to focus on the quality of the reconstruction, and therefore the quality of the answers generated by the model, or to focus on speed.

7 Limitations

The main limitation of this experiment is that it evaluates randomly selected weights, and not a fully quantized model. This means that we can measure thge quality of the reconstruction, but it does not measure the full impact of quantizing the entirety of the Qwen3-0.6B model. However, doing so would require much more resources than what was available for the experiment. Also, the metric for quality would need to change, because we are not measuring reconstruction error

anymore, but actual logits. Pherhaps the KL Divergence can be used as metric in that case, but the experiment escapes the resources available at this point.

8 Conclusion

This experiment compared four 2-bit quantization algorithms on actual Qwen3-0.6B weight blocks under a controlled paired design.

It consisted on creating a pilot experiment, to determine the sample size. Later, an ANOVA experiment was performend, and after realizing that the quantize and dequantize metrics were not normal, a friedman test was performed as well. Both yielded significant differences for the algorithms. Then, post-hoc comparisons were done to determine which one is best for each metric.

Leech Lattice Vector Quantization produced the best reconstruction quality, with SQNR mean 1.937 bits. QuIP# E8P12 was second best on reconstruction quality, with SQNR mean 1.784 bits. Uniform scalar quantization was the fastest method, with mean quantization time 0.000112 seconds and mean dequantization time 0.000050 seconds. QTIP bitshift did not dominate either quality or runtime in this experimental configuration.

Therefore, the recommended algorithm depends on the objective. If quality is the priority, Leech Lattice Vector Quantization should be selected. If runtime is the priority, especially for quantization and dequantization latency, other tecniques should be considered. In these measurements, no algorithm is best simultaneously for all three metrics.

References

[1] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. *QTIP: Quantization with Trellises and Incoherence Processing*. arXiv:2406.11235, 2024. `https://arxiv.org/abs/2406.11235`

[2] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. *QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks*. Proceedings of the 41st International Conference on Machine Learning, 2024. `https://arxiv.org/abs/2402.04396`

[3] Qwen Team. *Qwen3-0.6B*. Hugging Face model repository, 2025. `https://huggingface.co/Qwen/Qwen3-0.6B`

[4] Tycho F. A. van der Ouderaa, Mart van Baalen, Paul Whatmough, and Markus Nagel. *Leech Lattice Vector Quantization for Efficient LLM Compression*. arXiv:2603.11021, 2026. `https://arxiv.org/abs/2603.11021`